



NanoPulldown: a Local *In Silico* Pulldown Screen

AlphaPulldown-style bait/prey screening with ESMFold2-Fast on a single Apple
Silicon Mac Studio

Etienne Reboul

Groupe Gibbérellines et adaptation à l'environnement
Institut de Biologie Moléculaire des Plantes (IBMP), Strasbourg

October 9, 2026

Outline

1. Introduction

- Background
- Problem statement
- Language-model folding
- Project Outline

2. Pipeline

- Pipeline overview
- Input
- Preprocessing
- Folding
- Postprocessing

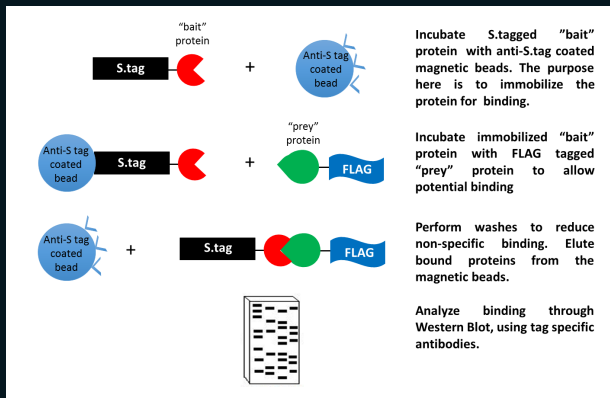
3. Conclusion & perspective

- Conclusion
- Perspective

Introduction

Pull-down assay: finding what binds a bait

Background



Pull-down with tagged proteins (S-tag / FLAG). RaihaT, Wikimedia Commons, CC

BY-SA 3.0

1. **Pull-down:** tagged *bait* immobilized on beads
2. Incubate with *prey* proteins (purified, or a lysate)
3. Wash, then elute
4. Readout: Western blot, or **mass spectrometry (IP-MS)** for a proteome-wide list of candidate preys

Limits: direct and indirect binders are mixed.

The AlphaFold2 breakthrough

Background

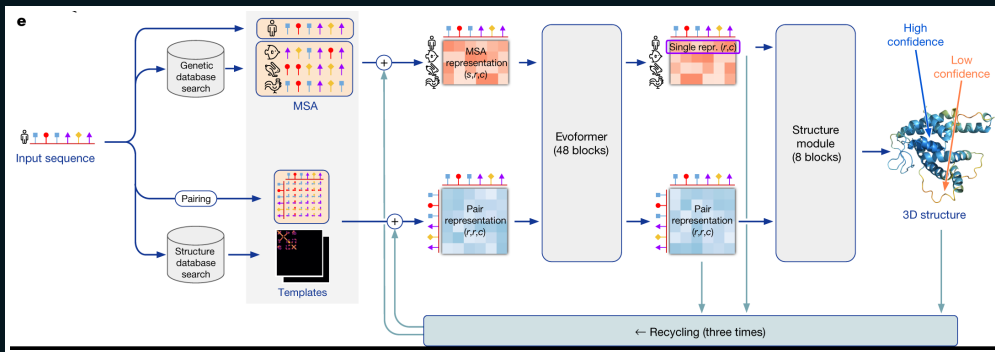


Fig. 1e, cropped; Wikimedia Commons, CC BY 4.0 (Jumper et al., 2021)

⇒ AF2 multimer is a **protein-protein docking engine** with confidence scores

John Jumper, Richard Evans, Alexander Pritzel, et al. **Highly accurate protein structure prediction with AlphaFold.**

Nature, 596:583–589, 2021.

URL <https://doi.org/10.1038/s41586-021-03819-2>

AlphaPulldown: the workflow

Background

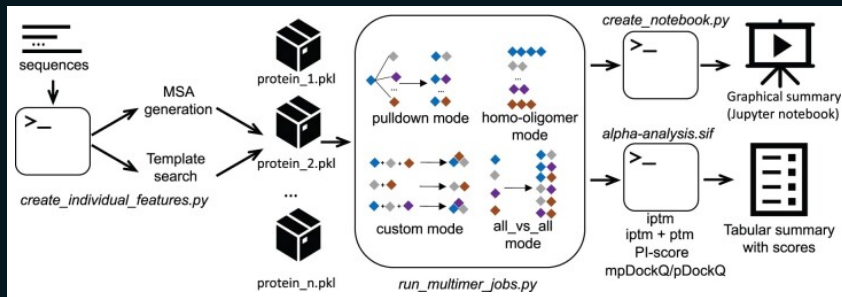


Fig. 1 of the paper; CC BY 4.0 (Yu et al., 2023)

⇒ AlphaPulldown automates identification of direct binding partners through the **in silico pulldown** with AF2-Multimer

Dingquan Yu, Grzegorz Chojnowski, Maria Rosenthal, and Jan Kosinski. [AlphaPulldown – a python package for protein–protein interaction screens using AlphaFold-Multimer](https://doi.org/10.1093/bioinformatics/btac749).

Bioinformatics, 39(1):btac749, 2023.

URL <https://doi.org/10.1093/bioinformatics/btac749>

Problem statement

Problem statement

- AlphaPulldown is **great**, but it assumes:
 - access to a GPU cluster :
 - ▶ Unmanageable to give access to a large number of users
 - ▶ Pangloss (ibmp cluster) does not have high-end GPUs for a full screen
 - ▶ High-end GPUs are expensive (around 12k EUR) currently not available through public procurement
 - MSA building :
 - ▶ Typically a slow process with Jackhmmer and HHblits
 - ▶ Requires large sequence databases (UniRef90, MGnify, BFD, PDB70, etc.)
 - ▶ Not feasible on a laptop
 - AlphaFold models as docking engine :
 - ▶ AF2 multimer is no longer State-of-the-Art (SOTA) for protein-protein docking
 - ▶ AF3 weights are available on demand but have specific licensing restrictions

⇒ Can we run the same screen on a single MacStudio (range price : 3k-9k EUR)?

Protein LM as alternative folding engines

Language-model folding

Protein LMs (e.g. ESMFold, Lin et al., 2023): **LLM-style transformers** trained on proteins.

	AF2-Multimer	ESMFold-type (protein LM)
Input	sequence + MSA + templates	single sequence
Evolutionary signal	read from the alignment at inference	learned in the weights during pre-training
Pre-processing	database searches (slow, heavy)	none
Weak spot	few homologs $\Rightarrow$ poor MSA	orphan / novel sequences

Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, et al. [Evolutionary-scale prediction of atomic-level protein structure with a language model](#).

Science, 379(6637):1123–1130, 2023.

URL <https://doi.org/10.1126/science.ade2574>

ESMFold2-Fast

Language-model folding

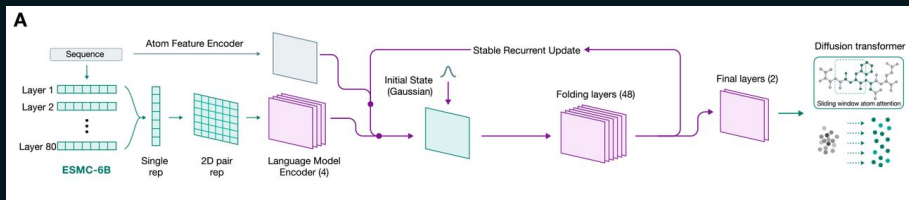


Fig. 2A, cropped, CC BY 4.0 (Candido et al., 2026)

- **ESMFold2-Fast** is designed for **high-throughput screening** and **design** of protein sequences
 - **Simplified architecture**: 24 folding layers instead of 48 to reduce memory footprint and inference time
 - **No MSA**: sequence-only prediction competitive with MSA-based methods (slightly worse than AF3)
 - ~9.4 s for 1024 residues on one H100 (reported 1.3× faster than AF3)
- ⇒ **This is what makes a high-throughput pulldown screen on a Mac Studio feasible**

Candido, Hayes, Derry, Rao, Lin, Verkuil, et al. *Language modeling materializes a world model of protein biology*. *bioRxiv*, 2026.

URL <https://doi.org/10.64898/2026.06.03.729735>

Goal

Project Outline

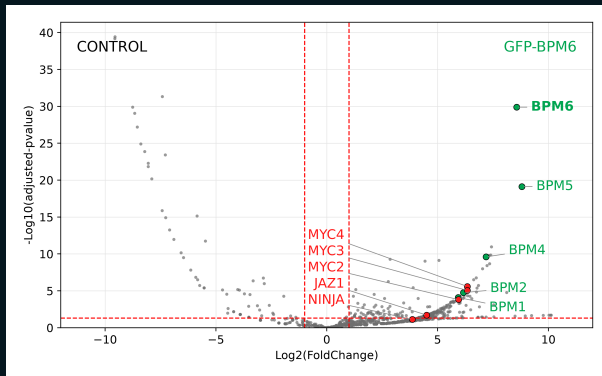
- Primary goal: Running a pulldown screen with ESMFold2-Fast on a single Apple Silicon Mac Studio
- Secondary technical goals:
 - Reproducible and scalable pipeline using Snakemake (workflow management system)
 - Easy-to-use configuration for adding new bait/prey pairs
 - Easy-to-run on a single machine (no cluster, no HPC scheduler)
 - Easy-to-read report with all the relevant metrics and figures
- Test case: BPM6 screen over the PXD013906 prey list and compare ranking against the PNAS Dataset S1 known / significant interactors (Chico et al., 2020)

Jose Manuel Chico, Esther Lechner, et al. *CUL3BPM e3 ubiquitin ligases regulate MYC2, MYC3, and MYC4 stability and JA responses. Proceedings of the National Academy of Sciences*, 117(11):6205–6215, 2020.
URL <https://doi.org/10.1073/pnas.1912199117>

Pipeline

Test case: what the BPM6 pulldown found

Pipeline overview



Reproduction of Fig. 1D from Dataset S1 data, CC BY (Chico et al., 2020)^a

Jose Manuel Chico, Esther Lechner, et al. CUL3BPM e3 ubiquitin ligases regulate MYC2, MYC3, and MYC4 stability and JA responses.

Proceedings of the National Academy of Sciences, 117(11):6205–6215, 2020.

URL <https://doi.org/10.1073/pnas.1912199117>

- **Experiment:** GFP-BPM6 immuno-precipitation from *A. thaliana* seedlings, LC-MS/MS (PXD013906)
- **Volcano plot:** $x = \log_2$ fold change (BPM6 / control), $y = -\log_{10}$ adjusted p (spectral counts, $n = 3$ vs 3)
- Dashed lines: $FC < 0.5$ or > 2 , adjusted $p < 0.05$
- **Green:** BPM paralogs (bait BPM6, BPM1/2/4/5) → heterodimers
- **Red:** MYC2, MYC3, MYC4, JAZ1, NINJA → candidate substrates

⇒ **Our benchmark:** can we find the same top interactants in the *in silico* screen?

Stage 1 : Preprocessing (local, needs internet)
UniProt fetch → domains (ScanProsite, InterPro) → disorder + MoRF (AIUPred)

Stage 2 : Folding (Apple Silicon GPU, MLX/Metal)
ESMFold2-Fast → metadata compression → OpenMM minimization

Rescoring & report (local)
ipSAE / iLIS / Pinc → PLIP contacts → reports/report.zip

Everything runs on one machine, in one Snakemake call.

Input: bait and prey lists

Input

```
$ head configs/preys.txt
```

```
AOA1I9LLD2
```

```
AOA1I9LMI7
```

```
AOA1I9LNN0
```

```
AOA1I9LPP1
```

```
AOA1I9LPQ2
```

```
AOA1I9LPZ1
```

```
AOA1I9LQ33
```

```
AOA1I9LR66
```

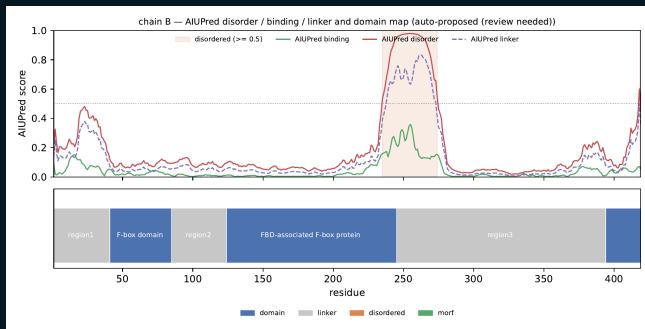
```
...
```

configs/preys.txt: one UniProt accession per line

- two text files with UniProt accessions:
 - configs/baits.txt → one bait per line
 - configs/preys.txt → one prey per line
 - a script generates all the bait × prey pairs config files (yaml)
- ⇒ **simple input files and one liner to add a new screen**

Preprocessing: domains, disorder and binding regions

Preprocessing



F4HR18 (prey of Q9M548): AIUPred disorder / binding / linker + domain map

- Automatic preprocessing of all unique sequences (baits + preys) : Domain annotation and IDR / MoRF prediction
 - **IDR**: intrinsically disordered regions (long, flexible)
 - **MoRF**: short disordered regions that fold upon binding
- ⇒ sequence are annotated to characterize the binding regions in interface analysis

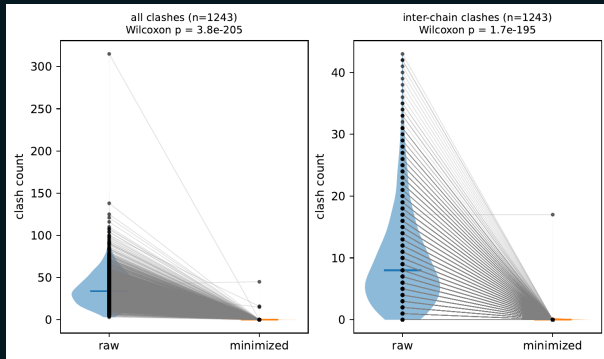
Folding with ESMFold2-Fast on Apple Silicon

Folding

- Folding with **ESMFold2-Fast** with lower computational requirements:
 - 10 loops
 - 100 sampling steps
 - 1 seed
 - 1 diffusion sample
- **Memory usage** is the main bottleneck for folding on Apple Silicon (M1/M2/M4)
- force a **Token limit**: peak memory grows $\sim n^{2.2}$; on a 36 GB M4 Max, 1 000 tokens $\approx$ 29 GB, 145 s (`max_tokens: 1000`)
- Metadata compression to reduce disk footprint

Minimization removes the steric clashes

Postprocessing

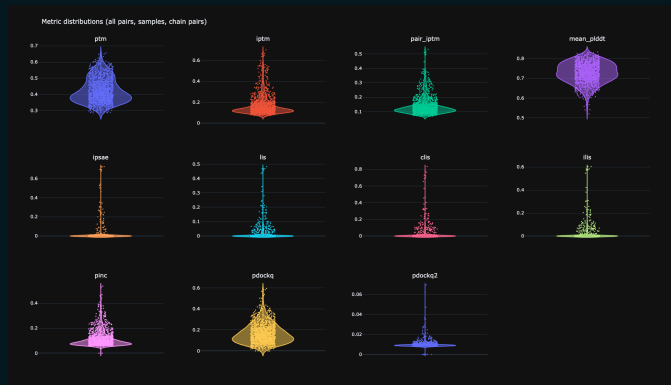


Clash count per model, raw vs. minimized (n=1243)

- Done with **OpenMM** (amber14 + GBn2 implicit solvent; OpenCL with CPU fallback)
 - Automatic Quality Control : check for divergence, NaN, or energy increase → discard the sample
 - Clashes drop to ≈ 0 for nearly all models, inter-chain included (Wilcoxon $p < 10^{-190}$)
- ⇒ **Prevent false positives induced by steric clashes in the contact analysis**

Rescoring the interface

Postprocessing

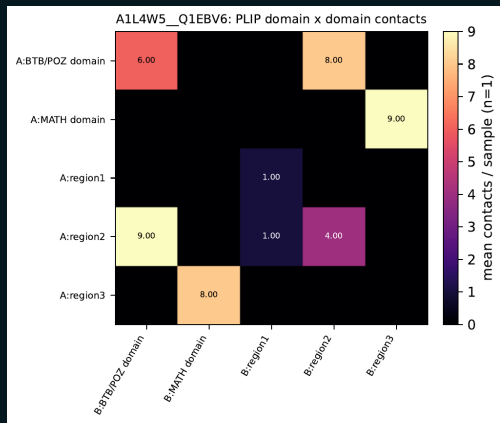


Distributions of the implemented SOTA interface metrics (ipSAE, iLS, Pinc, pair ipTM, ...) over all pairs, samples and chain pairs

⇒ Used to rank preys per bait

Characterizing the interface with PLIP

Postprocessing

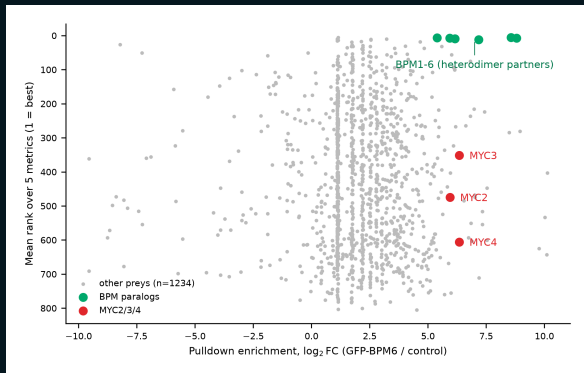


BPM6 × BPM5 (top interactor): PLIP domain × domain contacts

- **PLIP** : Protein-Ligand Interaction Profiler, detects non-covalent contacts in protein-protein interfaces
- Aggregated per **domain pair**, using the automatically annotated domains from the preprocessing stage
- **Contact types**: hydrogen bonds, hydrophobic contacts, salt bridges

In silico pulldown results vs. BPM6 pulldown

Postprocessing



1 243 BPM6 $\times$ prey pairs; enrichment from (Chico et al., 2020)

- **Pulldown:** MYC2/3/4 strongly enriched ($\log_2 FC \approx 6$)
- **Models:** mean rank MYC3 351, MYC2 474, MYC4 606; none in the top 50 on any metric; ipSAE = iLIS = 0 for all three
- PLIP contacts mid-pack (26–37, median 28); top counts: BPM paralogs
- **Possible explanation, untested:** BPM6 binds MYC2/3 only *very weakly* in yeast two-hybrid (Chico et al., 2020); the co-IP may be bridged by other BPMs
- **Caveats:** Y2H is limited by MYC/BPM toxicity in yeast; MBP pulldown still detects BPM6–MYC. Only BPM6 run as bait

One report, zipped

Postprocessing

- All the plots shown in the previous slides are generated automatically and saved in a single `report.zip` file
- Everything is packaged in a single **Zip** that is unpacked to html + figures + tables for easy browsing
- Contents:
 - domain / disorder / MoRF summary plots
 - results table across pairs
 - contact heatmaps
 - minimization energy plots
 - Statics about the runtime

Running it

Postprocessing

Running the pipeline is a single Snakemake command, after creating the conda env and generating the pair configs:

```
conda env create -f envs/controller.yaml
python scripts/expand_pairs.py      # baits x preys -> configs/<pair>.yaml
# add the new pair stems to config.yaml's pairs: list
snakemake --use-conda --cores 4 --resources gpu=1 docker_heavy=3
```

⇒ **Output:** one reports/report.zip

Conclusion & perspective

Conclusion

Conclusion

- A **local, MSA-free pulldown pipeline**: run on a single Apple Silicon Mac Studio, no cluster required
- A **complete workflow**: Preprocessing → ESMFold2-Fast → OpenMM → rescoring → PLIP, summarized in a single zipped report
- 1 243 BPM6 × prey pairs folded and minimized end to end in ~ 2.5 days on a M4 Max (36 GB) with 4 CPU cores + 1 GPU
- Adding a screen = editing the bait / prey lists + 2 or 3 commands

Perspective

Perspective

- Confirm with different bait / prey pairs, and with other known interactors
- An experimental validation of novel interactors is needed to confirm the predictions
- To expand the pipeline capabilities, we need better hardware (more GPU memory, more CPU cores) to run larger screens and longer sequences

Thank you for your attention



Team members

Patrick ACHARD
Jean-Michel DAVIÈRE
Lali ACHARD
Olivia LEDIEU
Hans RIOS
Antoine DAVID
Étienne REBOUL

IBMP bioinformatic facility

Valérie COGNAT
David PFIEGER
Stéphanie GRAINDORGE

Collaborators

Team: Rôle de l'ubiquitine dans la régulation cellulaire

Filippo BATTAGLIA
Cyriaque CRETON
Marie-Claire CRIQUI
Pascal GENSCHIK
Esther LECHNER
Sandra NOIR

Thomas POTUSCHAK
Marlène SCHIAFFINI
Wen Hui SHEN
Vishal VARSHNEY
Agathe VERCELLETTO



Questions?